API SECURITY

uygulama ve yazılım güvenliği

API güvenlik açığı içerebilir mi?

BOLA (Broken Object Level Authorization)

BOLA (Broken Object Level Authorization) Nedir?

Kullanıcının rolüne bağlı olarak hangi nesnelere erişebileceğinin belirlenilmesi ve yetkilendirilmesidir. Genel olarak BOLA verilerin korunması, yetkisiz erişimin engellenmesidir denilebilir.

```
// Güvenli Olmayan API Örneği
@GetMapping ( "/user/{id}" )
public User getUserProfile ( @PathVariable ( "id" ) String userId) {
    return  userService .getUserById (userId);   // Yetkilendirme kontrolü yok
 }
```

BOLA güvenlik açığı nasıl oluşturulur?

-Kullanıcı tarafından sağlanan girdilerin doğrulanmasında başarısızlık.

-Yetkilendirme kontrollerinin yanlış yapılandırılması

-URI'de, istek başlığında veya gövdesinde kolayca tanımlanabilir

veya numaralandırılabilir kaynak kimliklerini ortaya çıkarmak.

-Her bir istekte izinlerin kontrol edilmemesi

```
// Güvenli API Örneği
@GetMapping ( "/user/{id}" )
public  ResponseEntity < User > getUserProfile ( @PathVariable ( "id" ) String u
    User currentUser = ( User ) auth. getPrincipal ();
    if (!currentUser. getId (). equals (userId)) {
        return  ResponseEntity . status ( HttpStatus . FORBIDDEN ). build ();
    }
    return  ResponseEntity . ok (userService. getUserById (userId));
}
```

BOLA Saldırılarından Korunma Yöntemleri

01

Güvenlik Açığının Belirlenmesi

URL ve api uç noktalarının manipüle edile bilindiğinin fark edilmesiyle BOLA olduğu tespit edilir.

02

Uygun Erişim Kontrollerini Uygulayın

Nesnenin kullanım amacına uygun olarak erişim kontrollerinin belirlenmesi

03

Sağlam Kimlik Doğrulama ve Oturum Yönetimi Uygulayın

Belli bir nesneye kimin erişebileceği, değişimler yapabileceğinin kontrol edilmesi

04

Rastgele veya tahmin edilemez kimlikler kullanın.

Kullanılacak kimliklerin random, tahmin edilemez bir şekilde belirlenmesi

05

API Ağ Geçitlerinin ve Hız Sınırlamasının Uygulanması

Bir API ağ geçidi kullanılması ve bu şekilde kısa sürede fazla istek uygulanmasının önüne geçilmesi

Broken Authentication

Authentication Nedir ?

Kişinin sisteme giriş yaparken iddia ettiği kişi olup olmadığını kanıtlamasıdır.

Broken Authentication nedir?

API'lardaki hatalı ya da eksik kimlik doğrulamadan kaynaklı olarak saldırganların sisteme erişmesidir.

Broken Authentication güvenlik açığı nasıl oluşturulur?

-token doğrulamasının yetersiz yapılması

- tahmin edilebilir veya süresi dolmamış oturum bilgileriyle sistemde kalınabilmesi

- güvenli olmayan algoritmalarla imzalanmış JSON Web Token (JWT) kullanımı

Broken Authentication Saldırılarından Korunma Yöntemleri

URL'lerde Oturum Kimliklerinin Gösterilmemelidir

Oturum kimlikleri URL'lerde yer almamalıdır çünkü URL'ye erişimi olan herkes oturuma devam edebilir.

Uygun Oturum Süresini Ayarlayın

Web uygulamarı kullanılan oturumda belli bir süre etkinlik olmaması halinde oturumu kapatır, bu süreler uygulamaya göre özelleştirilmelidir.

Oturum Kimliklerini Döndür ve Geçersiz Kıl

Çalınan bir kimliğin uzun süre kullanılmasını engellemek amacıyla kimlikler belli periyotlarla değiştirilmedir.

Brute-Force Saldırılarına Karşı Koruma Uygulayın

Saldırganlar doğru kimlik doğrulamasını bulmak için farklı kombinasyonlar deneyebilirler. Bunun için başarısız giriş denemeleri sınırlandırılmalıdır.

Multi-Factor Authentication

Birden fazla kimlik doğrulama yöntemi gerektiren bir güvenlik sistemi kullanılmalıdır.

random session token

Oturum yönetimi için elden geldiğince “random session token” üretilmesi ve “session token” larının doğru kullanımı ve hatta uygun oturum yönetim uygulamalarının seçimi sağlanmalıdır.

BOPLA (Broken Object Property Level Authorization)

BOPLA (Broken Object Property Level Authorization) nedir?

BOPLA, aşırı veri ifşası durumunda ortaya çıkan Bozuk Nesne Özellik Düzeyi Yetkilendirmesi anlamına gelir.Bu güvenlik açığını tespit etmek zordur çünkü basit kimlik doğrulama kontrollerinden daha fazlasını içerir ve iş mantığına, belirli alanlara/nesnelere ve kötü niyetli olarak hazırlanmış isteklere odaklanır.
Bu tür bir güvenlik açığı genellikle REST API'lerinde ve GraphQL'de bulunur; burada iç içe nesneler temel özellik düzeyindeki kontrollerden geçmez ve bilgileri olduğu gibi döndürür.

BOPLA ve BOLA arasındaki fark nedir?

BOLA nesnenin kendisine erişimi sorunuyken BOPLA nesnenin özelliklerine erişimi zafiyetirdir.. BOLA'DA başkasına ait nesneye erişebilirken BOPLA' da nesneye ait ve izin verilmemiş alanları değiştirebilir ya da görebilir.

Legitimate request – user retrieving stored credit card information

Request:

```
POST /payments/storedcard/json HTTP/1.1

Host: payments.host.com
Connection: close
Content-Length: 78
Cache-Control: max-age=0
Origin: https://payments.host.com
Content-Type:
application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/87.0.4280.88
Safari/537.36
Accept:
text/html,application/xhtml+xml,applicat
ion/xml;q=0.9,image/avif,image/webp,imag
e/apng,*/*;q=0.8,application/signed-exch
ange;v=b3;q=0.9
Referer:
https://payments.host.com/methods/
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Cookie:
session=kjasdnfklnf01922wndlasdjkfnz-0f7
1k9013u18901u2mklsduhz12234d4D
```

Response with data exposure - HTTP response to the API call contains sensitive data in the message body

Response:

```
HTTP/1.1 200 OK
Cache-Control: no-cache, no-store,
max-age=0, must-revalidate
Pragma: no-cache
Expires: Mon, 01 Jan 1990 00:00:00 GMT
Date: Wed, 27 Jan 2021 15:43:39 GMT
Content-Type: application/json;
charset=utf-8
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Connection: close
Content-Length: 55

[{"PAN":"4111111123454321","status":"ok"
,"CVV":"1234"}]
```

Legitimate - Client sends a legitimate request

```
PUT /api/v2/users/5deb9097 HTTP/1.1

User-Agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/78.0.3904.108
Safari/537.36
X-Forwarded-For: 19.42.129.253

{
  "_id": "5deb9097",
  "address": "******, NY City, NY",
  "company_role": "Investment Services",
  "email": "******",
  "first_name": "******",
  "full_name": "******",
  "job_title": "Broker",
  "last_name": "******",
  "phone_number": "******"
}
```

Attack – Attackers sends the same request but adds the admin role in the request body

```
PUT /api/v2/users/5deb9097 HTTP/1.1

User-Agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/78.0.3904.108
Safari/537.36
X-Forwarded-For: 19.42.129.253

{
  "_id": "5deb9097",
  "address": "******, NY City, NY",
  "company_role": "Investment Services",
  "email": "******",
  "first_name": "******",
  "full_name": "******",
  "is_admin": true,
  "is_sso": true,
  "job_title": "Broker",
  "last_name": "******",
  "permission_type": "admin",
  "phone_number": "******",
  "role": "admin",
  "sso_type": "admin",
  "system_user_type": "admin",
  "system_user_type_cd": 2,
  "user_type": "admin",
  "user_type_cd": 10
}
```

BOPLA Saldırılarından Korunma Yöntemleri

01

Yetki Kontrolü

Kullanıcı yalnızca kendi verisine erişebilsin.

02

Güvenli Veri Dönüşümü

Genel to_json() yerine izinli alanları döndür.

03

Kontrollü Girdi

Otomatik nesne bağlamayı kısıtla, whitelist kullan.

04

Değişim Yetkisi

Sadece yetkili kullanıcı veri değiştirebilsin.

05

Yanıt Doğrulama

API çıktısını şemaya göre kontrol et.

Unrestricted Resource Consumption

Unrestricted resource consumption nedir?

API’larda kısıtlamanın yetersiz olması ve saldırganın aşırı ya da hatalı istekler göndererek maliyet artışına, hizmet kesintilerine ve potansiyel güvenlik açıklarına sebep olan bir zafiyettir.

BOLA ve unrestricted resource consumption arasındaki bağlantı:

Bir API'de BOLA mevcut olduğunda, saldırganlar erişim izni olmayan kaynakları tüketmek için bunu istismar edebilir ve bu da hizmet reddine yol açabilir.

Sınırsız Veri Yükleme

```
@app.route('/upload', methods=['POST'])
def upload():
file = request.files['file']
file.save(f'/path/to/save/{file.filename}')
return 'File uploaded successfully', 200
```

Hız Sınırlaması Yok

```
@app.route('/expensive-operation', methods=['GET'])
def expensive_operation():
# Simulate an expensive operation like fetching a large dataset
result = {"data": [i for i in range(1000000)]} # Large response
return jsonify(result)
```

Unrestricted Resource Consumption Saldırılarından Korunma Yöntemleri

Kullanım Politikaları

Açık ve net kötüye kullanım politikaları belirleyin.
API kullanım sınırlarını ve ihlal durumundaki yaptırımları tanımlayın.

01

Erişim Kontrolü

Rol ve yetkilere dayalı erişim kontrolü uygulayın.
Yetkisiz erişimi ve gereksiz kaynak tüketimini engelleyin.

02

Kaynak & İstek Sınırları

Kullanıcı başına kaynak limitleri belirleyin.
Rate limiting ile belirli süredeki istek sayısını kısıtlayın.

03

Güvenli Entegrasyon

Üçüncü taraf API bağlantılarını güvenli ve kontrollü yönetin.
Hassas servislere doğrudan erişimi sınırlandırın.

04

İzleme & Harcama Kontrolü

API trafiğini gerçek zamanlı izleyin.
Şüpheli aktiviteleri tespit edin ve harcama limitleri belirleyin.

05

BFLA (Broken Function Level Authorization)

Broken Function Level Authorization nedir?

Kullanıcıların rolüne göre erişmesi gereken fonksiyonların doğru şekilde kontrol edilmemesi sonucu oluşan güvenlik açığıdır. Saldırganlar yetkileri olmayan yönetici veya özel işlemlere erişebilir.

Broken Function Level Authorization nasıl oluşur?

-Rol ve yetki kontrollerinin eksik yapılması

-Yönetici işlemlerinin normal kullanıcılar tarafından çağrılabilmesi

-Endpoint’lerin rol bazlı kontrol edilmemesi

-API fonksiyonlarının yanlış yapılandırılması

Legitimate – POST method is correctly requested	Attack – Request is modified to send a DELETE method
POST /example/api/v1/provision/user/search HTTP/1.1 User-Agent: AHC/1.0	DELETE /example/api/v1/provision/user/search HTTP/1.1 User-Agent: AHC/1.0

Connection: keep-alive
Accept: */*
Content-Type: application/json;
charset=UTF-8
Content-Length: 131
X-Forwarded-For: 10.93.23.4
 	
{
"search_filter":
"user_id=exampleId_100",
"max_return": "250",
"page_size": "250",
"return_attributes": [
 	
]
}
	Connection: keep-alive
	Accept: */*
	Content-Type: application/json;
	charset=UTF-8
	Content-Length: 131
	X-Forwarded-For: 10.93.23.4
	
	{
	"search_filter":
	"user_id=exampleId_100",
	"max_return": “250”,
	"page_size": “250”,
	"return_attributes": [
	
	]
	}

BFLA Korunma Yöntemleri

01

Fonksiyon Düzeyinde Doğrulama

Kullanıcıların yalnızca yetkili oldukları API fonksiyonlarına erişebilmesini sağlar.

02

Güvenli Oturum Yönetimi

Oturumların güvenli yönetimi yetkisiz erişimi ve oturum ele geçirilmesini önler.

03

Yönetimsel Erişim Kontrolü

Yönetici işlemlerine erişim rol ve ek güvenlik kontrolleri ile sınırlandırılmalıdır.

04

API Güvenlik Testleri

API güvenlik açıklarını tespit etmek için düzenli güvenlik testleri yapılmalıdır.

UASBF (Unrestricted Access to Sensitive Business Flows)

Unrestricted Access to Sensitive Business Flows nedir?

API’deki bazı kritik işlemlerin (satın alma, yorum gönderme, hesap oluşturma gibi) yeterli sınırlandırma olmadan çalıştırılmasıdır. Bu durum saldırganların otomatik istekler göndererek sistemi kötüye kullanmasına ve hizmetin zarar görmesine neden olabilir.

Unrestricted Access to Sensitive Business Flows nasıl oluşur?

-Rate limiting uygulanmaması

-Otomatik isteklerin engellenmemesi

-CAPTCHA veya bot korumasının olmaması

-İş mantığı kontrollerinin eksik olması

UASBF Korunma Yöntemleri

Güçlü güvenlik programları uygulayın

Güvenlik açıklarını düzenli olarak tespit edin, raporlayın ve giderin.

Kullanıcı davranışlarını izleyin

Şüpheli ve otomatik API kullanımını tespit etmek için izleme mekanizmaları kullanın.

Rate limiting uygulayın

Belirli bir süre içinde yapılabilecek API isteklerini sınırlandırarak kötüye kullanımı önleyin.

Erişim kontrolleri kullanın

Hassas API fonksiyonlarına yalnızca yetkili kullanıcıların erişmesini sağlayın.

Düzenli güvenlik testleri yapın

API uç noktalarını ve işlevlerini sürekli olarak test ederek güvenlik açıklarını erken tespit edin.

API ağ geçidi ve güvenlik önlemleri kullanın

Trafiği analiz ederek zararlı istekleri engelleyin.

SSRF (Server Side Request Forgery)

Server Side Request Forgery nedir?

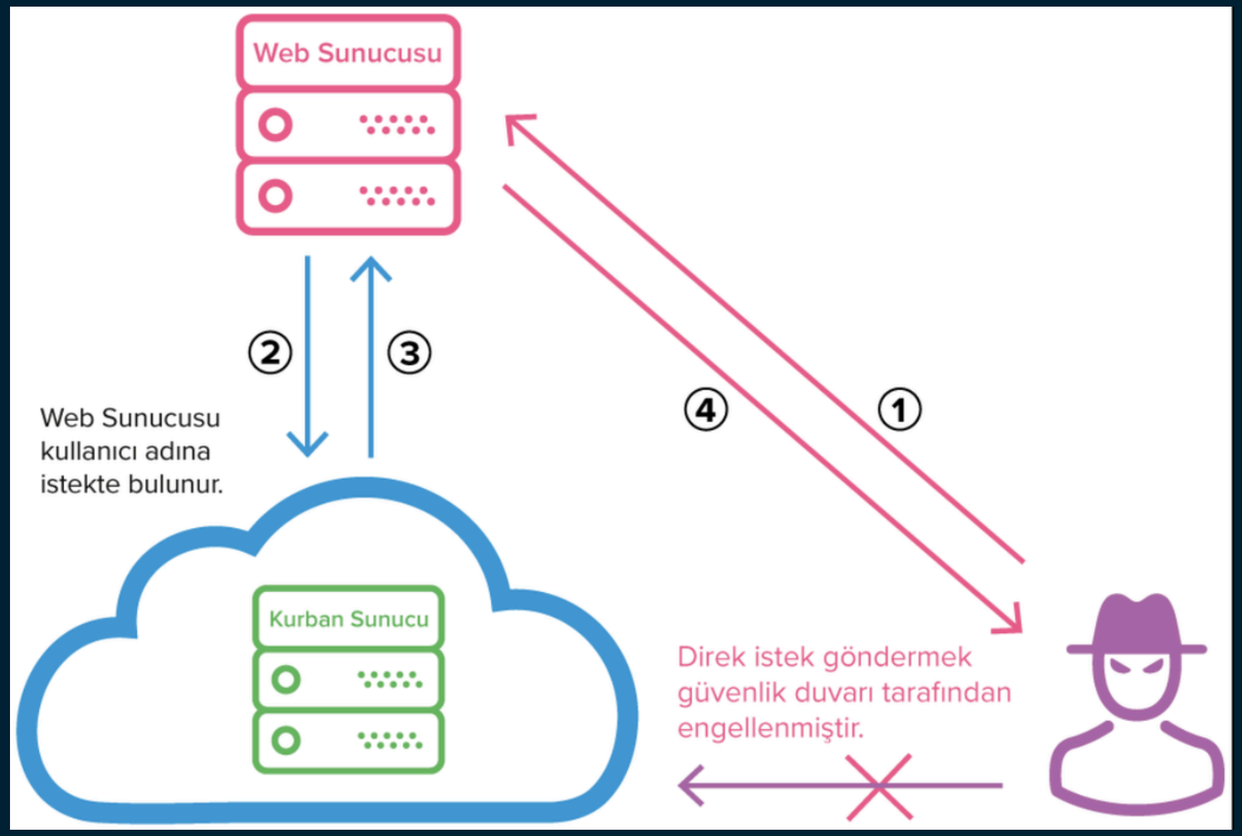
API'nin kullanıcıdan aldığı bir URL'ye doğrulama yapmadan istek göndermesi, sonucu oluşan güvenlik açığıdır. Bu durum saldırganların sunucuyu kullanarak iç ağdaki sistemlere veya yetkisiz kaynaklara erişmesine yol açabilir.

Server Side Request Forgery nasıl oluşur?

-Kullanıcı tarafından verilen URL'nin doğrulanmaması

-İç ağ adreslerine erişimin engellenmemesi

-Güvenlik duvarı veya ağ filtrelerinin eksik olması



SSRF Korunma Yöntemleri

Protokolleri Kısıtlayın

Uygulamanın yalnızca HTTP ve HTTPS protokollerini kullanmasını sağlayın

İzin Listesi (Allowlist) Uygulayın

Yalnızca güvenilir alan adlarına (domain) ve IP adreslerine erişim izni verin. Beklenmeyen tüm uzak kaynak taleplerini varsayılan olarak engelleyin.

İç Ağ İzolasyonu (Segmentation)

unucunun localhost (127.0.0.1) ve bulut metadata servisleri (169.254.169.254) gibi hassas iç kaynaklara erişimini güvenlik duvarı ile kesin.

Giriş Doğrulama ve Filtreleme

Kullanıcıdan gelen URL ve IP girdilerini sıkı bir şekilde doğrulayın. Girdileri normalize ederek özel karakterleri ve manipülasyon denemelerini filtreleyin.

Yanıt Kontrolü ve Maskeleme

Dış kaynaklardan gelen yanıtların içeriğini kontrol edin ve ham veriyi asla kullanıcıya döndürmeyin. Hata mesajlarında iç ağ bilgilerini gizleyin.

Security Misconfiguration

Security Misconfiguration nedir?

API veya sunucu ayarlarının yanlış ya da eksik yapılandırılması sonucu ortaya çıkan güvenlik açıklarıdır. Varsayılan ayarlar, açık servisler veya hatalı güvenlik ayarları saldırganların sistemi istismar etmesine neden olabilir.

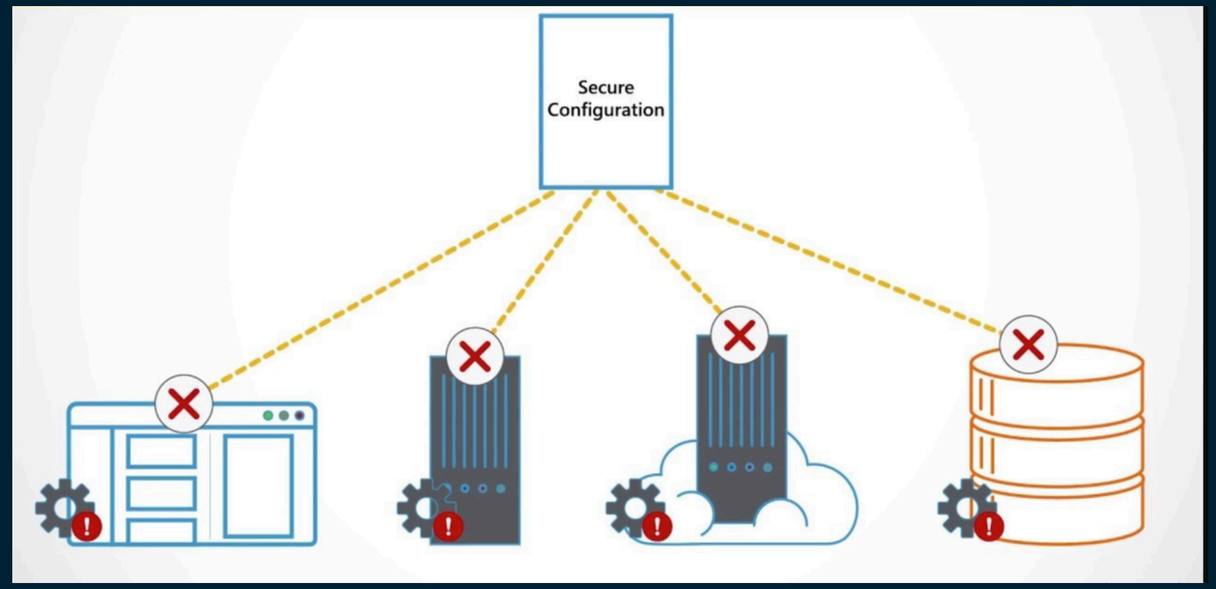
Security Misconfiguration nasıl oluşur?

-Hata ayıklama (debug) modunun üretimde açık unutulması.

-Gereksiz servis, port ve özelliklerin aktif bırakılması.

-Bulut depolama ve API izinlerinin herkese açık olması.

-Güvenlik başlıklarının (headers) eksik yapılandırılması.



Security Misconfiguration Korunma Yöntemleri

Sıkılaştırma (Hardening) ve Temizlik

Kullanılmayan servis ve portları kapatın; varsayılan şifreleri kurulum sonrası mutlaka değiştirin.

01

Güncelleme Yönetimi

Yazılım, kütüphane ve framework'leri güncel tutun; güvenlik yamalarını geciktirmeden uygulayın.

02

Bilgi Gizleme

Detaylı hata mesajlarını ve sistem bilgilerini (stack trace) gizleyin; sadece genel mesajlar verin.

03

Otomatize Denetim

Düzenli zafiyet taramaları ve otomatik konfigürasyon kontrolleri ile güvenliği sürekli kılın.

04

Improper Inventory Management

Improper Inventory Management nedir?

API’lerin, sürümlerinin ve uç noktalarının doğru şekilde takip edilmemesi durumudur.
Eski veya unutulmuş API’lerin açık kalması saldırganların bu zayıf noktaları kullanmasına neden olabilir.

Improper Inventory Management nasıl oluşur?

-Varsayılan şifre ve ayarların değiştirilmemesi.

-Güncel olmayan yazılım ve kütüphanelerin kullanımı.

-Kayıt dışı ve dokümante edilmemiş gizli (shadow) uç noktaların sistemde unutulması.

-Güvenlik desteği kesilmiş eski sürüm (v1, v2) API'lerin hala yayında tutulması.

-API'lerin hangi verilere eriştiğine dair güncel bir envanter kaydının bulunmaması.

-Test veya hazırlık (staging) ortamlarının canlı sistemlerde (production) açık bırakılması.

-Hızla büyüyen sistemlerde hangi servislerin aktif olduğunun takip edilememesi.

Improper Inventory Management Korunma Yöntemleri

Güncel Envanter ve Dokümantasyon

Tüm API uç noktalarını listeleyin ve Swagger / OpenAPI gibi araçlarla güncel dokümantasyon tutun.

Sürüm Kontrolü (Versioning)

Eski API sürümlerini (v1, v2) düzenli olarak emekliye ayırın ve yalnızca güncel sürümleri yayında tutun.

Ortam Ayrıştırması

Üretim ortamındaki API'leri, test ve geliştirme aşamasındaki "gölge" uç noktalardan tamamen izole edin.

Sürekli Keşif ve Tarama

Envanter dışı kalmış, unutulmuş API'leri tespit etmek için düzenli otomatik taramalar gerçekleştirin.

Unsafe Consumption of APIs

Unsafe Consumption of APIs nedir?

Bir uygulamanın üçüncü taraf API’lerden gelen verilere yeterli güvenlik kontrolü yapmadan güvenmesidir.
Bu durum hatalı veya kötü niyetli verilerin sisteme girmesine ve güvenlik risklerinin oluşmasına yol açabilir.

Unsafe Consumption of APIs nasıl oluşur?

-Diğer API'lerden gelen verilerin güvenilir kabul edilip doğrulanmadan işlenmesi.

-Üçüncü taraf servislerle kurulan bağlantılarda HTTPS yerine güvensiz protokollerin kullanılması.

-Dış servislerden gelen yönlendirmelerin (redirects) kontrolsüz bir şekilde takip edilmesi.

-API anahtarlarının ve kimlik bilgilerinin kod içerisinde açık veya güvensiz saklanması.

-Dış servislerden dönen verilerin doğrudan veri tabanına veya kullanıcı arayüzüne aktarılması.

Unsafe Consumption of APIs Korunma Yöntemleri

Girdi Doğrulaması

Dış API'lerden gelen her türlü veriyi "kullanıcı girdisi" gibi değerlendirin; işlemeden önce mutlaka doğrulayın ve filtreleyin.

Güvenli İletişim

Tüm API entegrasyonlarında TLS/HTTPS protokollerini zorunlu tutun ve sertifika doğrulaması yapın.

En Az Yetki Prensibi

Üçüncü taraf servisler için tanımlanan API anahtarlarına yalnızca gerekli olan minimum yetkileri tanımlayın.

Hata ve Zaman Aşımı Yönetimi

Dış servislerin çökmesi veya yavaşlamasına karşı "timeout" limitleri belirleyin; hata mesajlarının sistem bilgilerini sızdırmasını engelleyin.

dinlediğiniz için teşekkür ederim...